```
use Symfony\Component\HttpFoundation\JsonResponse;

// automatically JSON encode assoc. array, set correct media-type header
$response = new JsonResponse(
    ['data' => 'will be encoded automatically']
);
```

… or redirecting the client:

```
use Symfony\Component\HttpFoundation\RedirectResponse;

// respond with 302 Found status code and Location: header
$response = new RedirectResponse('http://example.com/');
```

Full documentation can be found at https://symfony.com/doc/current/components/http_foundation.html

Sessions

HTTP is "stateless" by design, which means that every request must independently contain enough information to fulfill that request, and there is no mechanism within the protocol itself to retain any information about the state of a particular connection between one request and another. On the other hand, a web application will likely want to store information about its users, for at least as long as the browser window (or tab) contains the site. The concept of a user making a sequence of related requests to the same web server within a timeframe has a name: *session*. Although the HTTP protocol itself does not assume any persistence of state, there is no barrier to doing this within the web application itself. Web browsers have various options for persisting data, such as the `localStorage` and `sessionStorage` APIs – see https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage and https://developer.mozilla.org/en-US/docs/Web/API/Window/sessionStorage for more information. And of course, there's the very first solution to the problem, which is the Cookie API.

We've already looked at cookies, which were invented to allow the persistence of arbitrary data (based on domain and resource paths) between HTTP requests, but they have their limits. In particular, there are usually limits on the size of a single header – remember that cookies are passed in a single `Cookie:` header – and that limit is not